UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CIENCIAS

Almacenes y Minería de Datos

Tarea 3: Reglas de Asociación y Clasificación

Alumnos:

- Méndez Ávila Luis Geovanni 317143980
- Paredes Zamudio Luis Daniel 318159926

Profesora: L. en C.C. Jessica Santizo Galicia

04 de Mayo de 2026

1. Apriori

1.1. Preprocesamiento de Datos

Primeramente, haremos modificaciones para a los valores de las fechas (nacimiento, fecha de desaparición, fecha de registro) para pasarlos a un valor `datetime`. Esto nos ayudará a poder discretizar los valores de edad, mes y año de desaparición. En particular, con edad haremos una segunda discretización para trabajar con grupos de edad (0-12, 13-17, 18-30, 31-50 y 50+).

```
# Convertir fechas a datetime
datos['FECHA_NACIMIENTO'] = pd.to_datetime(datos['FECHA_NACIMIENTO'], format='%Y-%m-%d',
    errors='coerce')
datos['FECHA_DESAPARICION'] = pd.to_datetime(datos['FECHA_DESAPARICION'], format='%Y-%m-%d %
    H:%M:%S', errors='coerce')
datos['FECHA_REGISTRO'] = pd.to_datetime(datos['FECHA_REGISTRO'], format='%Y-%m-%d %H:%M:%S'
    , errors='coerce')

# Discretizar EDAD
datos['EDAD'] = (pd.Timestamp.now() - datos['FECHA_NACIMIENTO']).dt.days / 365.25
datos['GRUPO_EDAD'] = pd.cut(datos['EDAD'], bins=[0, 12, 18, 30, 50, 100],
                             labels=['0-12', '13-17', '18-30', '31-50', '50+'])

# Discretizar FECHA_DESAPARICION
datos['MES_DESAPARICION'] = datos['FECHA_DESAPARICION'].dt.month
datos['AÑO_DESAPARICION'] = datos['FECHA_DESAPARICION'].dt.year
```

Luego, removimos las columnas originales de donde viene esta información para poder trabajar con lo que ya se tiene discretizado y aquellas que no contienen información que nos sea de utilidad para ocupar los algoritmos.

```
columnas_a_remover = [
    'ID_VICTIMA',
    'CVE_ENT',
    'CVE_MUN',
    'FECHA_NACIMIENTO',
    'FECHA_DESAPARICION',
    'FECHA_REGISTRO',
]

datos_limpio = datos.drop(columns=columnas_a_remover, errors='ignore')
datos_limpio = datos_limpio[~(datos_limpio == 'CONFIDENCIAL').all(axis=1)]
```

Algo que notamos con una revisión manual del dataset es que existen muchas entradas con un valor 'CONFIDENCIAL'. Decidimos dejar estos valores para poder llevar a una interpretación de los mismos posteriormente.

Este dataset modificado será usado tanto en nuestra implementación de Apriori como usando la de `mlxtend`.

1.2. Implementación Propia de Apriori

Para nuestra implementación lo que realizamos fue crear una clase (y posteriormente un objeto) `Apriori` que incluyera los métodos necesarios para llevar a la ejecución del algortimo (cálculo de lift, de confianza, etc.). En general, usamos `frozensets` para tener conjuntos inmutables y qué de esta manera fuera más fácil eliminar duplicados, trabajar con subconjuntos iguales sin orden ({A, B} siendo lo mismo que {B, A}) y que nuestra ejecución fuera más optima a largo plazo.

Decidimos incluir dos cosas extra dentro del mismo objeto Apriori. La primera fue la funcionalidad para crear las reglas de asociación. Esto para que, al momento de leer el código, fuera más directo entender de donde venían dichas reglas (pues se invocaría haciendo `Apriori.genera_reglas_asociacion()`). La segunda fue la capacidad de realizar el cálculo de soporte individual para poder obtener el valor más cercano a usar en el algoritmo (resultando en este caso 0.05).

Como nota aparte, por nuestras pruebas manuales, decidimos ocupar un valor de confianza mínima de 0.3 en el algoritmo pues los resultados tenían más sentido a la realidad.

Nota

El código de esta implementación se encuentra disponible en este archivo en GitHub.

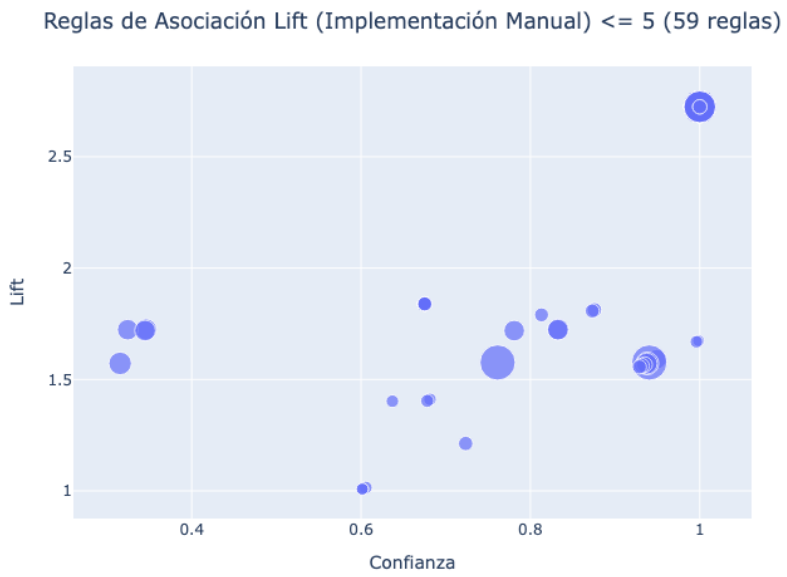
Mientras que el de la ejecución se ve a continuación:

```
apriori_manual = Apriori(soporte_min=0.05, confianza_min=0.3)
apriori_manual.carga_transacciones(datos_limpio, columnas=datos_limpio.columns)
itemsets_manual = apriori_manual.apriori()
reglas_asociacion = apriori_manual.genera_reglas_asociacion()
df_reglas_manual = apriori_manual.obtiene_reglas_dataframe()
# Guardado como .csv para procesamiento posterior
df_reglas_manual.to_csv('datos_apriori_manual.csv', encoding='utf-8')
```

Esta ejecución, en promedio, tomó 3.5 minutos en cada vuelta. Por que en nuestro caso, no fue la implementación más optima.

1.2.1. Resultados Obtenidos

Se obtuvieron en total 68 reglas de asociación con esta implementación, con un soporte mínimo de 0.05 y una confianza mínima de 0.3. Sin embargo, era necesario hacer una primera selección de las mismas, ya que algunas tienen un lift < 5, lo cual nos lleva a considerarlas como reglas espurias. Esto nos hace eliminar las primeras 9 reglas resultantes, con una distribución de lift que se muestra como sigue:



Una muestra de las reglas obtenidas se muestra a continuación:

```
,antecedente,consecuente,soporte,confianza,lift
10,"ENTIDAD=JALISCO', 'ESTATUS_VICTIMA=CONFIDENCIAL', 'MUNICIPIO=CONFIDENCIAL",SEXO=
    CONFIDENCIAL,0.0746898503962296,1.0,2.7241042544100598
11,"ENTIDAD=JALISCO', 'SEXO=CONFIDENCIAL",ESTATUS_VICTIMA=CONFIDENCIAL
    ,0.0746898503962296,1.0,2.7241042544100598
12,"ENTIDAD=JALISCO', 'SEXO=CONFIDENCIAL', 'MUNICIPIO=CONFIDENCIAL",ESTATUS_VICTIMA=
    CONFIDENCIAL,0.0746898503962296,1.0,2.7241042544100598
13,ESTATUS_VICTIMA=CONFIDENCIAL,"SEXO=CONFIDENCIAL', 'MUNICIPIO=CONFIDENCIAL
    ",0.3670931457124291,1.0,2.7241042544100598
14,SEXO=CONFIDENCIAL,MUNICIPIO=CONFIDENCIAL,0.3670931457124291,1.0,2.7241042544100598
15,"ENTIDAD=JALISCO', 'SEXO=CONFIDENCIAL",MUNICIPIO=CONFIDENCIAL
    ,0.0746898503962296,1.0,2.7241042544100598
16,MUNICIPIO=CONFIDENCIAL,SEXO=CONFIDENCIAL,0.3670931457124291,1.0,2.7241042544100598
17,ESTATUS_VICTIMA=CONFIDENCIAL,SEXO=CONFIDENCIAL,0.3670931457124291,1.0,2.7241042544100598
18,SEXO=CONFIDENCIAL,ESTATUS_VICTIMA=CONFIDENCIAL,0.3670931457124291,1.0,2.7241042544100598
19,MUNICIPIO=CONFIDENCIAL,ESTATUS_VICTIMA=CONFIDENCIAL
    ,0.3670931457124291,1.0,2.7241042544100598
20,ESTATUS_VICTIMA=CONFIDENCIAL,MUNICIPIO=CONFIDENCIAL
    ,0.3670931457124291,1.0,2.7241042544100598
```

Esta pre-selección de 59 reglas será usada posteriormente para comparar resultados con los obtenidos en la siguiente sección.

Nota

El archivo de las 59 reglas seleccionadas se encuentra disponible en este archivo en GitHub.

1.3. Usando Apriori de `mlxtend`

Para poder trabajar con `mlxtend` debemos hacer algunas modificaciones a una copia de nuestro dataset limpio (el que usamos anteriormente para ocupar nuestra implementación de Apriori). El dataframe que ocupa `mlxtend.aprori` debe ser un dataframe booleano o *onehot*. Esto es, un dataframe en donde se de convertir cada valor posible de una columna en una columna booleana. Por ejemplo, si tenemos que los valores en la columna 'SEXO' son HOMBRE, MUJER, etc, entonces se debe detener un columna SEXO=HOMBRE con valores True o False, otra con SEXO=MUJER, etc, etc.

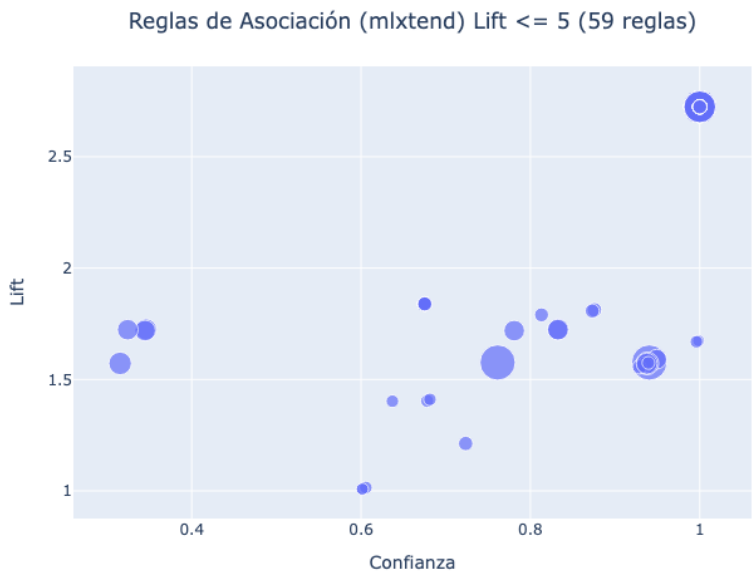
El código de esta implementación se ve de la siguiente manera:

```
# Para trabajar con algo que entienda el apriori de mlxtend, se debe convertir a un dataframe booleano/onehot.
# Esto es, convertir cada valor en una columna booleana.

datos_para_mlxtend = datos_limpio.copy()

# En lugar de insertar columna por columna, crear una lista de columnas
columnas_onehot = {}

for columna in datos_para_mlxtend.columns:
    valores_unicos = datos_para_mlxtend[columna].dropna().unique()

    for valor in valores_unicos:
        nombre_col = f"{columna}={valor}"
        columnas_onehot[nombre_col] = (datos_para_mlxtend[columna] == valor).astype(bool)

datos_onehot = pd.DataFrame(columnas_onehot)

itemsets_frequentes_mlxtend = apriori(datos_onehot, min_support=0.05, use_colnames=True)
reglas_mlxtend = association_rules(itemsets_frequentes_mlxtend, metric='confidence', min_threshold=0.3)
reglas_mlxtend = reglas_mlxtend[['antecedents', 'consequents', 'support', 'confidence', 'lift']]
reglas_mlxtend.to_csv('datos_mlxtend.csv', encoding='utf-8')
```

Esta ejecución en promedio tomó 35 segundos segundos en cada vuelta, lo que lo hace considerablemente más rápido y mejor optimizado que nuestra implementación.

1.3.1. Resultados Obtenidos

Sin embargo, aunque las reglas de asociación obtenidas con este método estuvieron en un orden distinto, obtuvimos las mismas 68 reglas de asociación que con nuestra implementación manual. De igual manera, se hace una preselección de reglas con lift< 5 obteniendo, de igual manera, 59 reglas de este conjunto He aquí una muestra de los resultados:




```
,antecedents,consequents,support,confidence,lift
0,ORIGEN_REPORTE=PROCURADURIA GENERAL DE JUSTICIA DEL ESTADO DE TAMAULIPAS,ESTATUS_VICTIMA=
    DESAPARECIDA,0.0542995212380589,0.6015721969383533,1.0082835308957736
7,SEXO=CONFIDENCIAL,ESTATUS_VICTIMA=CONFIDENCIAL,0.3670931457124291,1.0,2.7241042544100598
8,ESTATUS_VICTIMA=CONFIDENCIAL,SEXO=CONFIDENCIAL,0.3670931457124291,1.0,2.7241042544100598
9,ENTIDAD=JALISCO,SEXO=CONFIDENCIAL,0.0746898503962296,0.6751738572682466,1.8392439770508808
10,SEXO=CONFIDENCIAL,MUNICIPIO=CONFIDENCIAL,0.3670931457124291,1.0,2.7241042544100598
11,MUNICIPIO=CONFIDENCIAL,SEXO=CONFIDENCIAL,0.3670931457124291,1.0,2.7241042544100598
12,SEXO=HOMBRE,ESTATUS_VICTIMA=DESAPARECIDA
    ,0.4541889802594725,0.940370519283704,1.5761368500060997
```

Nota

El archivo de las 59 reglas usando mlxtend se encuentra disponible en este archivo en GitHub.

En consecuente, en lugar de hacer una comparativa de las reglas obtenidas, procedemos con la selección de las mismas para llegar a la interpretación de los resultados.

1.4. Interpretación de las Reglas de Asociación

1.5. Rangos de edad

Tenemos reglas que nos indican que los rangos de edad más comunes entre las víctimas desaparecidas.

ID	Antecedente	Consecuente	Soporte	Confianza	Lift
24	GRUPO_EDAD=50+	ESTATUS_VICTIMA=DESAPARECIDA	0.0840	0.9314	1.5611
25	GRUPO_EDAD=31-50	ESTATUS_VICTIMA=DESAPARECIDA	0.1881	0.3154	1.5723
26	ESTATUS_VICTIMA=DESAPARECIDA	GRUPO_EDAD=31-50	0.1882	0.3154	1.5723
27	GRUPO_EDAD=18-30	ESTATUS_VICTIMA=DESAPARECIDA	0.0874	0.9348	0.9348

Tabla 1: Regla directa para rango de edad condicionado a estatus desaparecida. Valores redondeados.

Estas reglas sugieren que el rango de edad promedio de las víctimas desaparecidas es entre 31 y 50 años (con valores de confianza de 0.3154). Esto puede indicar que las personas en este rango de edad son más vulnerables a la desaparición, o que hay un sesgo en el registro hacia estos grupos demográficos. Sin embargo, también es importante considerar que el rango de edad de 18-30 años tiene una confianza más alta (0.9348) para el estatus de desaparecida, lo que podría indicar que aunque hay menos casos en este rango, es más probable que sean reportados como desaparecidos.

1.6. Perfil Demográfico

Las siguientes reglas sugieren un perfil demográfico: conforme aumenta la edad, se observa una mayor sobrerrepresentación de hombres en el registro:

ID	Antecedente	Consecuente	Soporte	Confianza	Lift
14	GRUPO_EDAD=50+	SEXO=HOMBRE	0.0790	0.8756	1.813
15	GRUPO_EDAD=31-50	SEXO=HOMBRE	0.1671	0.8330	1.725
17	GRUPO_EDAD=18-30	SEXO=HOMBRE	0.0634	0.6781	1.404
53	SEXO=HOMBRE, ESTATUS_VICTIMA=DESAPARECIDA	GRUPO_EDAD=31-50	0.1567	0.3449	1.719

Tabla 2: Reglas de asociación para perfil demográfico (edad y sexo). Valores redondeados.

Aquí, los lifts crecientes (< 1) por grupo sugieren un sesgo de reporte/captura por edad y sexo.

1.7. Confidencialidad en Bloque

Estas reglas muestran un patrón sistémico: cuando un campo es confidencial, los demás también lo son, con una confianza de 1.0, en una proporción muy grande del dataset.

Esto nos lleva a esta conclusión: existe un mecanismo de censura sistémico de la información. Se realiza el registro de la persona desaparecida, pero no se deciden hacer públicos ninguno de los datos personales. Esto puede deberse a políticas de privacidad (sin explicar cuales son), temor a represalias, o falta de información capturada en el momento del registro.

ID	Antecedente	Consecuente	Support	Conf.	Lift
7	ESTATUS_VICTIMA=CONFIDENCIAL	SEXO=CONFIDENCIAL	0.3671	1.0000	2.724
10	SEXO=CONFIDENCIAL	MUNICIPIO=CONFIDENCIAL	0.3671	1.0000	2.724
20	ESTATUS_VICTIMA=CONFIDENCIAL	MUNICIPIO=CONFIDENCIAL	0.3671	1.0000	2.724

Tabla 3: Sesgo de registro: confidencialidad en bloque (censura/datos faltantes estructurados). Valores redondeados.

1.8. Sesgo por entidad — caso Jalisco

Estas reglas son un triste ejemplo de lo anterior. Si el registro de desaparición se hace en Jalisco, es muy probable que el resto de los campos terminen siendo confidenciales, lo que impide cualquier análisis de perfil o tendencia en esta entidad.

ID	Antecedente	Consecuente	Soporte	Confianza	Lift
9	ENTIDAD=JALISCO	SEXO=CONFIDENCIAL	0.0747	0.6752	1.839
19	ENTIDAD=JALISCO	ESTATUS_VICTIMA=CONFIDENCIAL	0.0747	0.6752	1.839
30	ENTIDAD=JALISCO	MUNICIPIO=CONFIDENCIAL	0.0747	0.6752	1.839
36	ESTATUS_VICTIMA=CONFIDENCIAL, ENTIDAD=JALISCO	SEXO=CONFIDENCIAL	0.0747	1.0000	2.724
45	ENTIDAD=JALISCO', 'SEXO=CONFIDENCIAL	MUNICIPIO=CONFIDENCIAL	0.0747	1.0000	2.724
46	ENTIDAD=JALISCO', 'MUNICIPIO=CONFIDENCIAL	SEXO=CONFIDENCIAL	0.0747	1.0000	2.724

Tabla 4: Sesgo por entidad: patrones de confidencialidad asociados a Jalisco. Valores redondeados.

1.9. Sesgo Temporal

En base a estas dos reglas podemos inferir que los registros de 2024 en adelante tienen un rezago de actualización, ya que no existe existe un cambio en el estado de la víctima. Esto se puede deber a que el proceso de investigación y actualización de casos es lento, o a que los casos más recientes aún no han sido actualizados en el sistema.

ID	Antecedente	Consecuente	Soporte	Confianza	Lift
28	AÑO_DESAPARICION=2025.0	ESTATUS_VICTIMA=DESAPARECIDA	0.0512	0.9985	1.674
29	AÑO_DESAPARICION=2024.0	ESTATUS_VICTIMA=DESAPARECIDA	0.0598	0.9960	1.669

Tabla 5: Sesgo temporal: asociación entre año reciente y estatus "DESAPARECIDA" (posible rezago de actualización). Valores redondeados.

2. Clasificación

El aprendizaje supervisado es una técnica donde un modelo aprende a partir de datos etiquetados. En este proyecto se desarrolló un modelo de clasificación supervisada utilizando el dataset proporcionado con el objetivo de predecir la variable `ESTATUS_VICTIMA`.

El objetivo principal fue construir un modelo capaz de identificar patrones en los datos y realizar predicciones sobre el estatus de una víctima utilizando diferentes variables del dataset.

Además de entrenar el modelo, también se realizó:

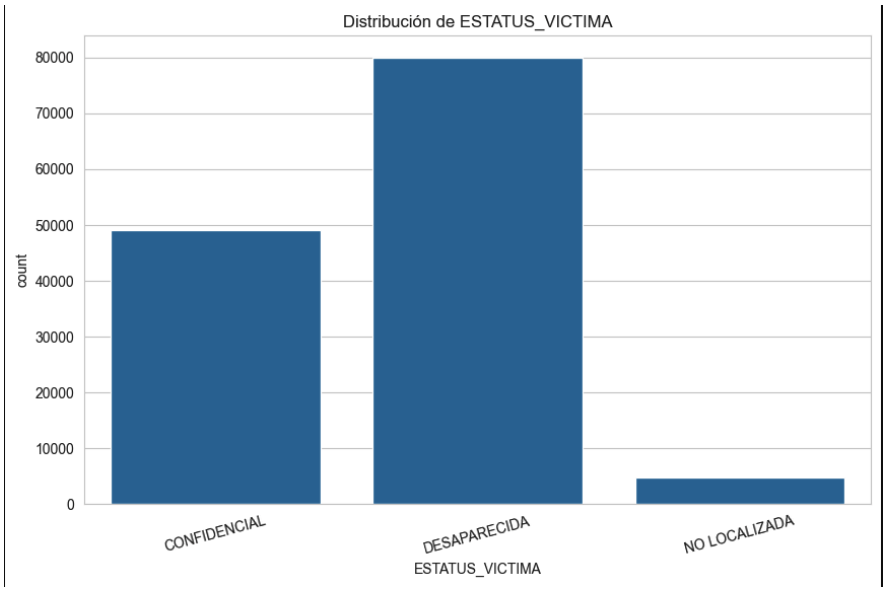
- División de entrenamiento y prueba.
- Evaluación mediante métricas de desempeño.
- Análisis de variables relevantes.
- Identificación de posibles sesgos en los datos.
- Relación de las métricas con escenarios reales.

2.1. Descripción del problema

El problema consiste realizar una clasificación multiclase, ya que la variable objetivo `ESTATUS_VICTIMA` puede contener diferentes categorías.

Entre las posibles clases se encuentran:

- DESAPARECIDA
- NO LOCALIZADA
- CONFIDENCIAL



El propósito del modelo es aprender las relaciones entre las variables predictoras y la variable objetivo para posteriormente realizar predicciones automáticas.

2.2. Preparación de los datos

Antes del entrenamiento del modelo fue necesario realizar una etapa de limpieza y preparación de datos.

Se realizaron las siguientes transformaciones:

- Eliminación de identificadores únicos como `ID_VICTIMA`, ya que no aportan información predictiva.
- Conversión de variables de fecha a formato datetime.
- Codificación de variables categóricas utilizando `LabelEncoder`.
- Manejo de valores faltantes mediante imputación.

Para evitar problemas durante el entrenamiento:

- Las variables categóricas fueron rellenadas con el valor `DESCONOCIDO`.
- Las variables numéricas fueron rellenadas con la mediana.

Esta estrategia ayuda a conservar información sin eliminar registros completos.

- 80 % de datos para entrenamiento.
- 20 % de datos para prueba.

La división se realizó utilizando `train_test_split()` de Scikit-learn.

Luego, se utilizó muestreo estratificado mediante el parámetro $stratify = y_encoded$, lo que garantiza que la proporción de clases se mantenga tanto en entrenamiento como en prueba.

Otra cosa a notar es que el dataset presenta desbalance de clases. Algunas categorías tienen muchos más registros que otras. Sin muestreo estratificado podrían ocurrir problemas como:

- Clases mal representadas.
- Evaluaciones poco confiables.
- Sesgo hacia la clase mayoritaria.

Por ello, el muestreo estratificado es importante para mantener una representación adecuada de todas las clases.

2.3. `Random Forest Classifier`

El modelo seleccionado para la clasificación fue el `Random Forest Classifier`, un algoritmo de aprendizaje supervisado basado en árboles de decisión. Se eligió este modelo debido a sus múltiples ventajas:

1. Maneja adecuadamente variables categóricas y relaciones no lineales.
2. Es robusto ante ruido y datos incompletos.
3. Reduce el sobreajuste en comparación con un árbol de decisión simple.
4. Funciona bien en datasets desbalanceados utilizando `class_weight='balanced'`.
5. Permite analizar la importancia de variables.
6. Ofrece un buen desempeño sin requerir ajustes extremadamente complejos.

Para evaluar el modelo se utilizaron las siguientes métricas y diagramas:

2.3.1. Accuracy

Para medir el porcentaje total de predicciones correctas y evaluar el desempeño general del modelo. Aunque proporciona una métrica sencilla de interpretar, en datasets desbalanceados puede resultar engañosa, ya que un modelo podría predecir siempre la clase mayoritaria y aún así obtener un accuracy alto.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

- TP = Verdaderos positivos.
- TN = Verdaderos negativos.
- FP = Falsos positivos.
- FN = Falsos negativos.

Un valor alto de Accuracy indica que el modelo clasifica correctamente una gran proporción de registros. Sin embargo, debido al desbalance de clases del dataset, este resultado debe interpretarse con cuidado.

Por ejemplo, si la mayoría de los registros pertenecen a una sola clase, el modelo podría obtener un Accuracy relativamente alto incluso si tiene dificultades para identificar correctamente las clases minoritarias.

2.3.2. Precision

$$Precision = \frac{TP}{TP + FP}$$

Para medir qué tan confiables son las predicciones positivas. Es importante cuando los falsos positivos generan consecuencias negativas. En un contexto real, clasificar incorrectamente un caso podría provocar:

- Uso innecesario de recursos.
- Investigaciones equivocadas.
- Información incorrecta.

Un valor alto de precisión significa que cuando el modelo predice una determinada categoría, generalmente tiene razón. En un escenario real esto es importante porque:

- Reduce falsas alarmas.
- Evita investigaciones innecesarias.
- Disminuye el uso incorrecto de recursos.

Si la precisión fuera baja, el modelo estaría clasificando incorrectamente muchos registros.

2.3.3. Recall

$$Recall = \frac{TP}{TP + FN}$$

En problemas relacionados con personas desaparecidas es importante minimizar los falsos negativos. Un falso negativo significa que el modelo no detectó correctamente un caso real. Esto puede implicar:

- Casos relevantes ignorados.
- Retrasos en procesos de búsqueda.
- Información incorrecta para la toma de decisiones.

Un recall alto significa que el modelo logra identificar la mayoría de los casos verdaderos. Si el Recall fuera bajo, significaría que el modelo está dejando pasar demasiados casos reales.

2.3.4. F1-Score

$$F1 - Score = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

El dataset presenta un desbalance de clases, por lo que el F1-Score es una métrica más adecuada para evaluar el desempeño del modelo, pues:

- Detecta correctamente casos positivos.
- Evita demasiadas falsas alarmas.

Por ello es una métrica más robusta que Accuracy en escenarios desbalanceados. Incluso, es un resumen del equilibrio entre Precision y Recall, lo que permite evaluar el desempeño del modelo de manera más completa.

2.3.5. Matriz de confusión

La matriz de confusión es una herramienta visual que muestra el desempeño del modelo en términos de verdaderos positivos, falsos positivos, verdaderos negativos y falsos negativos para cada clase. Permite identificar patrones de error específicos, como por ejemplo:

- Qué clases son clasificadas correctamente.
- Qué clases presentan más errores.
- Cuáles categorías son confundidas con mayor frecuencia.

Esto ayuda a identificar posibles problemas del modelo y entender mejor el comportamiento de las predicciones.

2.3.6. Importancia de variables

Una ventaja de Random Forest es que permite analizar la importancia de las variables. Las variables con mayor importancia son aquellas que más contribuyen a las decisiones del modelo. Entre las variables potencialmente relevantes se encuentran:

- Sexo.
- Entidad federativa.
- Edad o año de nacimiento.
- Fecha de desaparición.

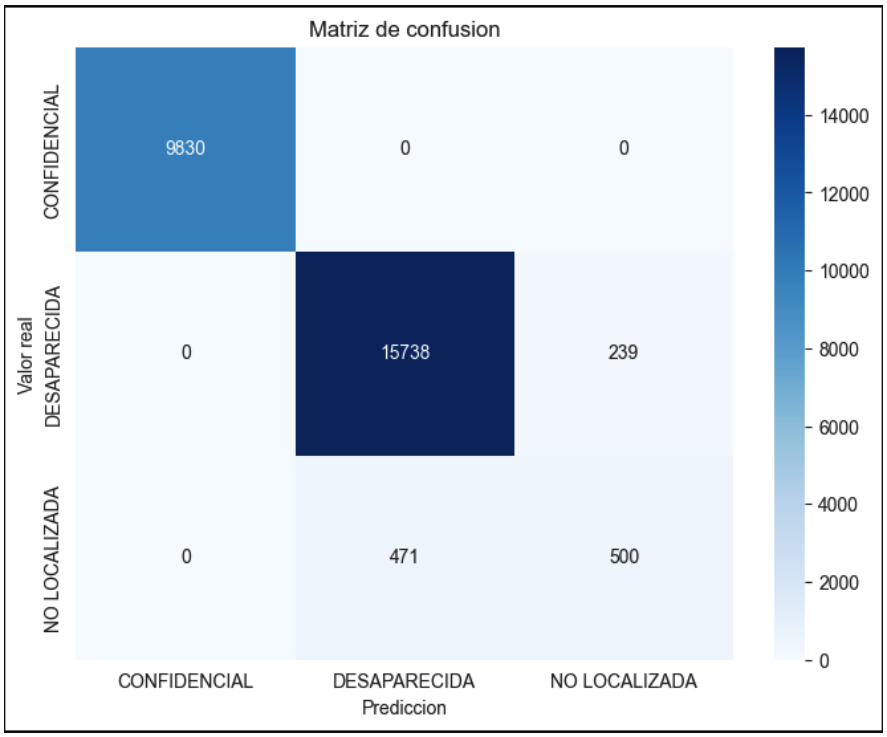
- Año de registro.

Estas variables pueden contener patrones útiles para la clasificación.

2.4. Resultados

```
Accuracy : 0.9735
Precision : 0.9709
Recall : 0.9735
F1-score : 0.9718
               precision    recall  f1-score   support

 CONFIDENCIAL       1.00      1.00      1.00      9830
 DESAPARECIDA       0.97      0.99      0.98     15977
NO LOCALIZADA       0.68      0.51      0.58       971

     accuracy                           0.97     26778
    macro avg       0.88      0.83      0.85     26778
 weighted avg       0.97      0.97      0.97     26778

```

Listing 1: Resultados del modelo



- El modelo muestra un buen desempeño general con un Accuracy del 97.35 %. Pero puede ser engañoso debido al desbalance de clases.

- La clase CONFIDENCIAL tiene una precisión y recall perfectos, lo que indica que el modelo clasifica correctamente todos los casos de esta categoría.
- La clase DESAPARECIDA también tiene un desempeño excelente con una precisión del 97 % y un recall del 99 %.
- Sin embargo, la clase NO LOCALIZADA presenta un desempeño más bajo, con una precisión del 68 % y un recall del 51 %, lo que indica que el modelo tiene dificultades para identificar correctamente esta categoría.
- El F1-score general es alto, pero es importante considerar, nuevamente, el desbalance de clases al interpretar estos resultados.
- La matriz de confusión muestra que la mayoría de los errores ocurren en la clase NO LOCALIZADA, lo que sugiere que el modelo tiene dificultades para distinguir esta categoría de las otra. (lo cual coincide con el desempeño bajo mencionado anteriormente).

A pesar de estas limitaciones, el modelo proporciona una aproximación útil para la clasificación de la variable `ESTATUS_VICTIMA`.

Nota

El cuaderno de Jupyter con el código completo del modelo de clasificación se encuentra disponible en el repositorio de GitHub del proyecto. o de manera directa en este enlace.